

USMA Theory and Mathematical Reference

This document is the canonical reference for **every numerical, statistical and signal-processing operation** performed by USMA. It is intended as the go-to reading whenever there is doubt about *what* a piece of code is doing to your data, *why*, and *what knob to turn* to influence it.

The document is organised top-down following the data flow:

1. Pixel extraction and signal reconstruction (raw $\rightarrow$ physical units)
 2. The two analysis methods on FRF/PSD signals (FFT, Lowpass)
 3. Coherence analysis
 4. Hit classification (how the four method votes become a verdict)
 5. The calibration engine (three estimators + hybrid merger)
 6. HSV colour calibration
 7. Graphical representations of every diagnostic plot
 8. Default parameter table
-

1. Signal reconstruction and Y-axis calibration

Source: `usma/analysis/signal.py` (`SignalAnalyzer._reconstruct_signal`, lines ~63–127).

Before any analysis runs, USMA must turn an HSV-masked ROI image into a one-dimensional signal in physical units (g/N, m/s², dimensionless coherence, etc.). This step is silent but it is the *foundation* on which everything else stands — if it is wrong, every threshold that follows is wrong.

1.1 From mask to pixel signal

The HSV mask (see §6) marks pixels belonging to the trace as white. For each column x of the ROI, USMA computes the *mean row* of all white pixels in that column. Columns with no white pixels are filled by linear interpolation from neighbouring columns.

```
y_coords, x_coords = np.nonzero(mask)
unique_x, idx      = np.unique(x_coords, return_inverse=True)
sum_y              = np.bincount(idx, weights=y_coords)
count_y           = np.bincount(idx)
anchor_y          = sum_y / count_y                                # mean row per column
signal_pixels      = np.interp(np.arange(width), unique_x, anchor_y)
signal_pixels      = height - signal_pixels                        # invert Y (image rows
grow downward)
```

The result, `signal_pixels`, has length equal to the ROI width. Each sample is a Y position measured in pixels from the bottom of the ROI.

1.2 From pixel signal to physical signal

USMA assumes the ROI's vertical axis is **linearly calibrated** between two user-supplied bounds, `region.y_axis_min` and `region.y_axis_max`. The conversion is a single affine map:

```
signal_physical[i] = y_axis_min + (signal_pixels[i] / height) * (y_axis_max -
y_axis_min)
```

Equivalently, if `pix_row[i]` is the original image row index:

```
signal_physical[i] = y_axis_min + ((height - pix_row[i]) / height) * (y_axis_max - y_axis_min)
```

Why this matters:

- **Pixel-space and physical-space are NOT the same signal.** They differ by a constant scale and offset, but that scale and offset propagate into every amplitude-based threshold (most importantly `residual_threshold`).
- **Spatial frequency in pixels is invariant** under the affine map, which is why the FFT method (§2) works on `signal_pixels` directly. It cares about the *shape* of the signal, not its amplitude.
- **The Lowpass method works on `signal_physical`** because its main control, `residual_threshold`, is an amplitude in real units. Mis-calibrated `y_axis_min/max` will silently shift the meaning of `residual_threshold`.

If you halve `y_axis_max - y_axis_min`, every residual is halved, and a `residual_threshold` that used to flag noise will now flag everything. **This is the single most common source of “calibration drift” complaints.**

1.3 Linear-axis assumption

USMA does **not** support log-Y axes. If the source plot uses a log scale, the reconstructed `signal_physical` is wrong by a non-linear factor and the LP threshold will be meaningless. The FFT method is partially shielded from this (it acts on shape, not amplitude) but its energy ratios will still be biased towards low frequencies. There is no autodetection — the user must ensure the source plot is linear.

The two analysis methods

Before diving into parameters, the architecture: every `frf` and `psd` ROI runs through `_analyze_wave_pattern()`, which performs two completely independent analyses on the same reconstructed pixel signal. Each produces a binary pass/fail result that feeds one of the four indicator lights.

FFT method

This operates on the **pixel-space signal** (raw column positions of the detected colour trace), not the physical-unit signal. The reason is that the frequency content of interest is a structural property of the *shape* of the signal, independent of what physical units it's in.

Steps: 1. Detrend: subtract the mean from `signal_pixels` 2. Apply `rfft()` → get complex spectrum 3. Compute magnitude spectrum `fft_mags = abs(yf)` 4. Build normalised frequency axis with `rfftfreq(N, 1)` → values from 0 to 0.5 (Nyquist)

FFT Cutoff Frequency (`fft_cutoff_frequency / psd_fft_cutoff_frequency`)

What it is: A normalised spatial frequency threshold, ranging from 0 to 0.5 (Nyquist = 0.5). The signal is *not* sampled in time — it's sampled in pixels horizontally across the ROI. So this is a **spatial frequency** in units of cycles/pixel, not Hz.

What it does: Splits the FFT spectrum into “low-frequency” ($\leq$ cutoff) and “high-frequency” ($>$ cutoff) halves. The code:

```
cutoff_indices = np.where(xf >= fft_cutoff)[0]
high_freq_energy = np.sum(fft_mags[cutoff_indices:] ** 2)
energy_ratio = high_freq_energy / total_energy
```

Intuition: A clean, smooth FRF/PSD signal changes slowly across the ROI $\rightarrow$ most energy at low spatial frequencies. A noisy or double-hit signal has rapid oscillations $\rightarrow$ elevated high-frequency energy. The cutoff defines the boundary between “slowly varying” and “rapidly oscillating.”

Default is 0.07, meaning anything above 7% of Nyquist is considered “high frequency.” With a typical 500-pixel-wide ROI, this corresponds to features that repeat every ~14 pixels or less.

Effect of changing it: Lower cutoff $\rightarrow$ more of the spectrum is labelled “high frequency” $\rightarrow$ higher energy ratios $\rightarrow$ easier to flag as BAD. Higher cutoff $\rightarrow$ stricter definition of “high frequency” $\rightarrow$ lower energy ratios $\rightarrow$ harder to flag.

FFT Energy Ratio Threshold (`fft_energy_ratio_threshold`)

What it is: The minimum `energy_ratio` to flag a signal as high-frequency. Dimensionless, ranges roughly 0.001–0.1 in practice.

What it does: `is_hf = energy_ratio > fft_threshold`. If the fraction of total spectral power above the cutoff exceeds this threshold, the FFT method votes BAD.

Default 0.006 = if more than 0.6% of the signal's total power is in high spatial frequencies, it's flagged. This is intentionally sensitive.

Lowpass method

This operates on the **physical-unit signal** (after converting pixel positions to real amplitude using the ROI's y-axis min/max calibration). It uses a Butterworth filter to separate smooth from rough components.

Steps: 1. Detrend: `subtract mean(signal_physical)` 2. Design a Butterworth low-pass filter: `butter(lp_order, lp_cutoff, btype='low')` 3. Apply zero-phase filtering: `filtfilt(b, a, signal_detrended)` 4. Compute residual: `residual = signal_detrended - filtered` 5. Count how many residual samples exceed `residual_threshold` 6. Compute `exceedance_ratio = exceedance_count / N` 7. Flag: `lowpass_is_bad = exceedance_ratio > exceedance_ratio_threshold`

LP Cutoff (`lowpass_cutoff` / `psd_lowpass_cutoff`)

What it is: The normalised cutoff frequency of the Butterworth filter. Same normalisation as the FFT: 0 to 1.0 (where 1.0 = Nyquist). In practice should stay below ~0.4 to remain meaningful.

What it does: Everything *below* this frequency passes through the filter unchanged. Everything *above* it is attenuated. The filtered signal represents the “smooth envelope” of the signal. The residual = what’s left over after removing the smooth component.

Intuition: Think of the lowpass filter as drawing a smooth curve through the signal. The residual is the jagged deviation between the actual signal and that smooth curve. A good hit has tiny residuals (the signal is smooth). A bad hit – rattling, double-hit, overload – has large residuals.

Effect of changing it: Lower cutoff → tighter smoothing → more of the signal ends up in the residual → more exceedances → easier to flag BAD. Higher cutoff → looser smoothing → only very rapid oscillations end up in the residual → fewer exceedances.

Default is 0.07 (same as FFT cutoff – this is a coincidence of the defaults, not a coupling).

LP Order (`lowpass_filter_order` / `psd_lowpass_filter_order`)

What it is: The order of the Butterworth filter passed to `scipy.signal.butter(order, cutoff, btype='low')`.

Why changing it seems to have no effect – this is the real answer to your question:

The Butterworth order controls the *steepness of the roll-off* in the transition band. At order 7 (the default), the filter is already very steep. The critical issue is that `filtfilt()` applies the filter **twice** – forward and backward – which squares the frequency response. So the effective order is $2 \times \text{lp_order}$. At order 7, the effective roll-off is the equivalent of a 14th-order filter. The transition band between “passes” and “stops” is already nearly a cliff.

This means: - **Changing order has almost no visible effect on the residual** because the filter is already sharp enough that the transition band is negligibly narrow - The residual is dominated by the *cutoff frequency* choice, not the order - Going from order 5 to order 9 at `lp_cutoff = 0.07` produces imperceptibly different residuals because at 0.07, the filter already attenuates the stopband to nearly zero at any reasonable order ≥ 3

There’s a secondary problem: `butter()` at high orders and low cutoff frequencies can produce numerically unstable filter coefficients (poorly conditioned `b`, `a` arrays). If you increase order too aggressively (≥ 9 –11) at low cutoffs like 0.07, `filtfilt` may silently diverge or produce NaN. The code has a try/except that falls back to the unfiltered signal in that case, which would make it appear as though the filter has no effect. The correct approach for robust high-order filtering would be `scipy.signal.sosfiltfilt()` with `butter(..., output='sos')`, which avoids `b`, `a` numerical instability entirely – this is a known issue worth fixing.

Practical guidance: Order is not a useful tuning knob at these settings. Leave it at 7. The effective controls for LP behaviour are `lp_cutoff` (sensitivity of what counts as smooth) and the two threshold parameters below.

Residual Threshold (`residual_threshold`)

What it is: An amplitude value in physical units (the same units as the signal — g/N, m/s², etc.). A residual sample is counted as an “exceedance” if `|residual[i]| > residual_threshold`.

What it does: Defines how large a deviation from the smooth envelope is considered noteworthy. Small value = strict, large value = permissive.

Default 0.005. The appropriate value depends entirely on the scale of your signal. A 1.0 g/N FRF will have residuals in a different range than a 0.01 g/N FRF. This is why calibration matters — the calibration engine sweeps this parameter to find the separation point between your Good and Bad distributions.

Exceedance Ratio Threshold (`exceedance_ratio_threshold`)

What it is: The minimum fraction of samples (0 to 1) that must exceed `residual_threshold` to flag the signal as BAD.

What it does: `lowpass_is_bad = exceedance_ratio > exc_threshold`. So if 0.7 (70%) of all samples have residuals larger than `residual_threshold`, it’s flagged BAD.

Default 0.7 is intentionally permissive — requiring 70% of the signal to be “rough” before flagging BAD. This means the LP method only fires on very noisy signals. If you want it more sensitive, lower this toward 0.2–0.3.

Parameter map summary—

Why LP Order seems to do nothing — deeper explanation

The core issue is that `filtfilt` gives you an **effective order of $2 \times \text{lp_order}$** , and Butterworth filters of order ≥ 5 are already very steep in their transition band. Here’s the roll-off magnitude at the cutoff frequency (by definition –3 dB for Butterworth), but what matters is how fast it falls *beyond* the cutoff:

Order	Effective order (filtfilt)	Attenuation at $2 \times$ cutoff
3	6	~–72 dB
5	10	~–120 dB
7	14	~–168 dB
9	18	~–216 dB

At the default `lp_cutoff = 0.07`, doubling to 0.14 is already in the deep stopband regardless of whether order is 5 or 9. The residual you measure is essentially identical.

The *real* risk of raising order is **numerical instability** in the `b`, a coefficient representation. `scipy.signal.butter()` at high orders and low cutoffs produces ill-conditioned polynomials. This can cause `filtfilt` to diverge silently, and the code’s `try/except` will catch it and return the unfiltered signal — which would appear in the graph as “LP has no effect at all.”

Recommended fix for a future DAILY.md: Replace the `butter(order, cutoff) → filtfilt` pattern with `butter(order, cutoff, output='sos') → sosfiltfilt`. Second-order sections (sos) are numerically stable at any order. This would also make the Order spinbox actually functional at higher values. This is a clean, low-risk change.

Practical tuning guide

Goal	Knob to turn
FFT method too sensitive (many false BADs)	Raise <code>fft_energy_ratio_threshold</code> or raise <code>fft_cutoff_frequency</code>
FFT method not sensitive enough	Lower <code>fft_energy_ratio_threshold</code> or lower <code>fft_cutoff_frequency</code>
LP method not firing at all	Lower <code>exceedance_ratio_threshold</code> (e.g. 0.7 → 0.3) or lower <code>residual_threshold</code>
LP method firing on clean hits	Raise <code>residual_threshold</code> (it may be miscalibrated for your signal scale)
LP Order	Leave at 7; change to <code>sosfiltfilt</code> before spending time on this

The most important insight: **LP Order is not a real tuning knob at the current implementation level.** The effective parameters controlling LP sensitivity are `lp_cutoff`, `residual_threshold`, and `exceedance_ratio_threshold`, in that order of impact.

3. Coherence analysis

Source: `usma/analysis/coherence.py` (`CoherenceAnalyzer.analyze`, lines ~20–92).

Coherence ROIs are treated separately from FRF/PSD ROIs because the coherence function $\gamma^2(f)$ lives in the bounded interval $[0, 1]$ and its degradation has a natural physical interpretation: how much of the response at each frequency is *not* explained by the input.

3.1 The badness integral

USMA inverts coherence and integrates it across the visible frequency band:

```

inverted_signal = 1.0 - signal_physical          # signal_physical is  $\gamma^2$  in
[0,1]
badness_integral = np.trapezoid(inverted_signal, freq_axis)
freq_span = max(region.x_axis_max - region.x_axis_min, 1.0)
normalized_badness = badness_integral / freq_span

```

`normalized_badness` is therefore the **average value of $(1 - \gamma^2)$** over the visible frequency range. It is dimensionless and lives in $[0, 1]$.

- 0.0 → perfect coherence everywhere

- 1.0 → zero coherence everywhere
- Typical “good” measurement → 0.05–0.15
- Anything above coherence_threshold (default 0.30) is flagged

The frequency-span normalisation is critical: it makes the metric **scale-invariant with respect to the visible band**. A user who zooms into a narrow band gets directly comparable numbers to a user looking at the full sweep, provided region.x_axis_min/max are correctly set.

3.2 Per-band badness

The same calculation is repeated on four equal-width sub-bands:

```
band_size = n // 4
for i in range(4):
    start_i = i * band_size
    end_i = (i + 1) * band_size if i < 3 else n
    band_inv = inverted_signal[start_i:end_i]
    band_freq = freq_axis[start_i:end_i]
    band_badness.append(
        np.trapezoid(band_inv, band_freq) / max(band_freq[-1] - band_freq[0],
1.0)
    )
```

This gives a 4-element vector that lets the GUI highlight *which* part of the spectrum is degrading — useful for spotting localised resonance/anti-resonance problems that an averaged metric would smooth out.

3.3 Run-level degradation

In addition to instantaneous badness, USMA tracks how badness evolves *within a measurement run*. If normalized_badness increases by more than coherence_degradation_pct (default 0.20 = 20 %) compared to the start of the run, the GUI raises a degradation warning. This is a lightweight trend detector, not a statistical test — it is purely a relative-change check.

Metric	Symbol	Formula	Default flag
Mean coherence	$\bar{\gamma}^2$	mean(signal_physical)	—
Min coherence	$\gamma^2_{\min}$	min(signal_physical)	—
Normalised badness	B	$\int (1 - \gamma^2) df / \Delta f$	> 0.30
Per-band badness	B _k	$\int_{\text{band}} (1 - \gamma^2) df / \Delta f_{\text{band}}$	—
Degradation	$\Delta B / B_0$	$(B - B_{\text{run_start}}) / B_{\text{run_start}}$	> 0.20

4. Hit classification

Source: usma/analysis/classifier.py (HitClassifier.classify_hit, lines ~10–56).

Each frame produces up to **four method votes**: FRF-FFT, FRF-LP, PSD-FFT, PSD-LP. Coherence is *not* part of the hit verdict — it is a separate measurement-quality channel. The classifier turns

those four binary votes into one of three labels, with each method individually toggleable in the config.

The logic, in plain words:

- Let `any_bad` = at least one enabled method voted BAD.
- Let `all_bad` = both the FRF side **and** the PSD side have at least one BAD vote (when both sides exist). If only one side exists, `all_bad` requires *both* methods on that side to be BAD.

Then:

- not `any_bad` → **GOOD HIT** (green)
- `all_bad` → **BAD HIT (...)** (red), with the parenthetical listing every failing method
- otherwise → **SUSPECT (...)** (orange), with the parenthetical listing every failing method

This means a single failing method on a single signal type is never enough to fail a hit outright — it always demotes it to SUSPECT first. A SUSPECT hit becomes a BAD hit only when **both** the FRF and PSD signal types independently agree there is a problem (or, when only one signal type exists, when both methods on that side agree).

The detail string concatenates failures as e.g. "FRF-FFT+PSD-LP", which is what shows up in the log and the indicator-light tooltip.

5. The calibration engine

Source: `usma/models.py` (CalibrationSignal, CalibrationSession, PercentileBoundaryEstimator, BayesianThresholdEstimator, ROCYoudenEstimator, HybridCalibrationEngine; lines ~161–845).

USMA's calibration system exists because the four threshold parameters described in §2 (`fft_cutoff_frequency`, `fft_energy_ratio_threshold`, `lowpass_cutoff`, `residual_threshold`, `exceedance_ratio_threshold`) cannot be set sensibly without a population of labelled signals. The calibration engine takes a stream of user-judged hits ("this one is GOOD", "this one is BAD") and infers the thresholds that best separate the two populations.

The architecture is deliberately **plural**: three independent estimators are run side-by-side and merged, because each one has different statistical assumptions and breaks down at different sample sizes. The merger is governed by an ordinal **confidence level** computed from the total number of labelled signals.

5.1 The data being calibrated

Each calibration sample is a `CalibrationSignal` containing:

- The reconstructed `signal_physical`
- Its precomputed FFT (`fft_freqs`, `fft_mags`)
- The current values of the metrics: `energy_ratio`, `exceedance_ratio`, `max_residual`
- A user judgment: "GOOD" or "BAD"

The estimators target three primary parameters:

- `fft_energy_ratio_threshold`
- `exceedance_ratio_threshold`
- `residual_threshold`

...and (only the percentile estimator) two filter-shape parameters:

- `fft_cutoff_frequency`
- `lowpass_cutoff`

5.2 Estimator 1 — `PercentileBoundaryEstimator`

Minimum data: ≥ 3 GOOD and ≥ 3 BAD.

Threshold rule (`_find_boundary`, line ~216):

```
max_good = max(good_values)
min_bad  = min(bad_values)

if min_bad > max_good:
    threshold = (max_good + min_bad) / 2.0      # cleanly separated
else:
    threshold = np.percentile(good_values, 95)  # overlapping → 5% FP rate on
GOOD
```

The intuition is direct: if the two populations don't overlap on a given metric, place the threshold at the midpoint between the highest GOOD and the lowest BAD. If they overlap, accept that some BADs will leak through but pin the threshold at the 95th percentile of the GOOD distribution so that no more than 5 % of *good* hits get incorrectly flagged.

Filter-shape sweep (`_sweep_filter_params`, lines ~232–305):

For `fft_cutoff_frequency`, USMA sweeps candidates over $[0.02, 0.25]$ in steps of 0.005. For each candidate it recomputes `energy_ratio` for every signal in the calibration set, then scores the candidate by **mean separation**:

```
good_mean = np.mean(ratios[labels == 1])
bad_mean  = np.mean(ratios[labels == 0])
separation = bad_mean - good_mean      # higher = better
```

The cutoff with the highest separation wins. The same procedure is applied to `lowpass_cutoff` over $[0.03, 0.20]$ in steps of 0.01, where the score is the separation in `exceedance_ratio` after re-running the LP residual computation.

This is why the Percentile estimator dominates the early phase of calibration: it is the only one that can *also* tune the filter shape, and it works with as few as 6 signals.

5.3 Estimator 2 — `BayesianThresholdEstimator`

Minimum data: ≥ 6 signals total (no per-class minimum).

Method: grid-based Bayesian inference with a sigmoid likelihood, one independent grid per parameter.

Grids (PARAM_GRIDS, line ~319):

Parameter	Range	Grid points
fft_energy_ratio_threshold	0.0005 – 0.15	300
exceedance_ratio_threshold	0.05 – 0.99	300
residual_threshold	0.0001 – 0.10	300

Prior: uniform over each grid ($\text{np.ones}(n) / n$). USMA does not assume any pre-existing belief about where the threshold lives.

Likelihood (lines ~333–380): each new signal updates the posterior using a sigmoid that asks “for each candidate threshold value t , how consistent is the new observation with that being the right cutoff?”

```
steepness = 20.0 / grid_range          # sharper for narrower ranges

if judgment == "GOOD":
    # GOOD signals should sit BELOW the threshold
    exponent = np.clip(-steepness * (grid - observed), -500, 500)
    likelihood = 1.0 / (1.0 + np.exp(exponent))
else:
    # BAD signals should sit ABOVE the threshold
    exponent = np.clip(-steepness * (observed - grid), -500, 500)
    likelihood = 1.0 / (1.0 + np.exp(exponent))

posterior = prior * likelihood
posterior /= posterior.sum()            # renormalise
```

Conceptually: a GOOD observation makes thresholds *above* the observed value more credible; a BAD observation makes thresholds *below* the observed value more credible. The sigmoid steepness controls how sharply the credibility flips around the observation.

Point estimate: the MAP (grid point with highest posterior).

Credible interval (lines ~399–404): from the posterior CDF,

```
cumulative = np.cumsum(posterior)
ci_low_idx = np.searchsorted(cumulative, 0.025)
ci_high_idx = np.searchsorted(cumulative, 0.975)
```

...giving a 95 % central credible interval that the GUI displays as a shaded band on the calibration plot. The width of this band is USMA’s principled measure of “how sure are we about this threshold yet”.

5.4 Estimator 3 — ROCYoudenEstimator

Minimum data: ≥ 5 GOOD and ≥ 5 BAD.

Method: for each metric, sweep candidate thresholds and pick the one that maximises Youden’s J statistic on the labelled data.

```
for thr in candidate_thresholds:
    tp = sum(1 for v in good_values if v <= thr)
    fp = sum(1 for v in bad_values if v <= thr)
    tpr = tp / n_good                # true-positive rate
    fpr = fp / n_bad                # false-positive rate
    J    = tpr - fpr                # Youden's J
```

The threshold maximising J is the optimal operating point on the ROC curve under the assumption that false positives and false negatives are equally costly.

AUC is computed alongside, by trapezoidal integration of the (FPR, TPR) pairs sorted by FPR:

```
auc = np.trapz(tprs_sorted, fprs_sorted)
auc = max(0.0, min(1.0, abs(auc)))
```

The AUC is reported per metric and is the headline number on the ROC subplot — it tells you whether the metric *itself* is even a good discriminator (AUC ~0.5 = no information; AUC > 0.9 = excellent).

5.5 The hybrid merger and confidence levels

HybridCalibrationEngine orchestrates everything and decides which estimators are eligible based on data volume.

Confidence level (lines ~562–574):

```
n = total_signals
if not meets_minimum: return 0
elif n <= 7:          return 1  # Percentile only
elif n <= 11:         return 2  # Percentile + Bayesian
elif n <= 15:         return 3  # All three, 20% agreement tolerance
else:                 return 4  # All three, 15% agreement tolerance
```

These cutoffs are **ordinal**, not probabilistic. Level 4 does not mean “90 % certain”; it means “we have ≥ 16 samples and can afford the strictest outlier rejection”.

Merge rules:

- **Level 1:** use Percentile estimates verbatim.
- **Level 2** (`_merge_two`, line ~601): weighted blend.

```
merged[key] = 0.4 * percentile[key] + 0.6 * bayesian[key]
```

The Bayesian estimate gets the heavier weight because at this sample size it has integrated more information per parameter than the percentile rule.

- **Level 3 / 4** (`_merge_all`, lines ~634–703): all three estimates are collected and an **agreement check** decides what to do.

```

mean_val = np.mean(estimates)
deviations = [abs(e - mean_val) / mean_val for e in estimates]

if all(d < agreement_pct for d in deviations):
    merged[key] = mean_val
else:
    outlier_idx = int(np.argmax(deviations))
    kept = [e for i, e in enumerate(estimates) if i != outlier_idx]
    merged[key] = float(np.mean(kept))

```

agreement_pct = 0.20 at Level 3, 0.15 at Level 4. So the engine first checks “do all three estimators agree to within 15–20 %?” — if yes, average them; if no, drop the most-deviant one and average the remaining two. This is a poor man’s robust mean: it tolerates one wild estimator without letting it pull the merged value.

5.6 What the engine does *not* do

A few things the engine deliberately does **not** attempt, so you don’t go looking for them:

- **No likelihood-ratio test** on whether GOOD and BAD distributions are actually distinguishable. The engine will happily produce thresholds even when the classes overlap completely; it is the user’s job to look at the calibration plot and notice this.
- **No correlated multivariate model.** All four metrics are calibrated independently; cross-parameter correlations (e.g. high energy ratios coinciding with high exceedance ratios) are visible in the plots but not exploited.
- **No online learning.** Each refit re-runs all three estimators from scratch on the full session. There is no incremental Bayesian update across sessions.
- **No automatic re-tuning of `fft_cutoff_frequency` or `lowpass_cutoff` past the initial fit.** Only the Percentile estimator sweeps these, and only when first run.

6. HSV colour calibration

Source: usma/gui/hsv_calibration.py, lines ~26–327.

Although it doesn’t look like a “math” topic, HSV calibration is a thresholding operation that determines which pixels make it into the signal at all (§1). Mistakes here propagate everywhere downstream.

6.1 The thresholding operation

USMA stores three lower bounds and three upper bounds:

```

hsv_lower = [H_min, S_min, V_min]
hsv_upper = [H_max, S_max, V_max]

```

The mask is computed by OpenCV’s per-channel `inRange`:

```

hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, np.array(hsv_lower), np.array(hsv_upper))

```

This is a logical AND across the three channels: a pixel is white in the mask iff $H_{\min} \leq H \leq H_{\max}$ **and** $S_{\min} \leq S \leq S_{\max}$ **and** $V_{\min} \leq V \leq V_{\max}$.

Channel ranges (OpenCV convention):

- Hue: 0–179 (note: 8-bit, NOT 0–360 — be careful when copy-pasting values from external HSV pickers)
- Saturation: 0–255
- Value: 0–255

Defaults (lines ~318–325): [0, 0, 0] to [179, 255, 240]. The upper Value is capped at 240 deliberately, to exclude pure white (which is what most plot backgrounds and gridlines are).

6.2 No auto-clustering

USMA’s HSV calibration is **purely interactive**. There is no k-means on the selected pixels, no Otsu-style automatic thresholding, no statistical estimation of Hue mode and spread. The user adjusts six sliders by hand and watches the live preview update. The reason is that plot traces are not a stationary distribution: anti-aliasing edges, gridline crossings and dashed segments mean that any data-driven estimator has to deal with several distinct sub-populations of pixels and tends to produce overly tight bounds. Manual tuning is faster and more reliable in this domain.

7. Graphical representations

Source: `usma/gui/graph_viewer.py` (per-signal plots and calibration diagnostics, lines ~17–373) and `usma/gui/hsv_calibration.py` (HSV preview, lines ~26–327).

This section maps every plot the GUI can show back to the math it visualises. If you ever look at a plot and aren’t sure what you’re seeing, look here.

7.1 Per-hit diagnostic plots

These are accessible from the graph viewer for any individual hit in history (`MAX_HISTORY = 25`).

Signal plot. Plain line plot of `signal_physical` (cyan) vs. pixel index. Y-axis label is the ROI’s `y_axis_unit`. This is the post-reconstruction signal in physical units — exactly what the LP method analyses.

FFT plot. Magnitude spectrum `fft_mags` (magenta) vs. normalised spatial frequency `fft_freqs` (0 to 0.5 Nyquist). Two overlays:

- A green dashed vertical line at `fft_cutoff_frequency`, marking the boundary between “low” and “high” spatial frequencies for the energy-ratio calculation.
- A red translucent fill under the curve **above** the cutoff, showing visually which part of the spectrum contributes to `high_freq_energy`.
- A text annotation "Energy ratio: <value>" so you can read it off without doing arithmetic.

Lowpass comparison. Two lines on the same axes:

- The original `signal_physical` (cyan)

- The Butterworth-filtered version (green dashed)

...with the gap between them lightly shaded. The shaded gap **is** the residual; this is the most intuitive way to see what the LP method is operating on.

Residual analysis. Standalone plot of `residual_physical` (orange) with two horizontal red dashed lines at `+residual_threshold` and `-residual_threshold`. Any sample whose magnitude exceeds the band is overlaid as a red bar. A text annotation reports "Exceedances: `<count>/<N>` (`<percentage>`)". The fraction is `exceedance_ratio`, and the LP verdict is BAD iff this fraction exceeds `exceedance_ratio_threshold`.

Run summary. Aggregated stats across all hits in the current run: counts of GOOD/SUSPECT/BAD, distributions of `energy_ratio` and `exceedance_ratio`, and (for coherence ROIs) the trend of `normalized_badness` over the run.

7.2 Calibration diagnostic plots

`_plot_calibration_diagnostics` (lines ~255–373) chooses one of three layouts depending on confidence level:

- **Level 1–2:** three side-by-side distribution plots, one per primary metric.
- **Level 3+:** a 2×2 grid adding an ROC subplot.
- **≥ 6 signals:** a convergence subplot is appended showing how each threshold has evolved as more samples were added.

Distribution subplots (one per metric). Two overlapping histograms drawn at `density=True` so they are comparable regardless of sample count:

- GOOD population in green (`#2ECC71`), `alpha = 0.6`
- BAD population in red (`#E74C3C`), `alpha = 0.6`

A solid white vertical line marks the **merged threshold** (i.e. whatever the hybrid engine produced after running through §5.5). When the Bayesian estimator has a posterior, a translucent blue band (`#3498DB`, `alpha = 0.15`) spans the **95 % credible interval** [`ci_low`, `ci_high`]. A narrow blue band means the Bayesian posterior is sharp; a wide band means more samples are needed before the threshold can be trusted.

How to read these:

- Two well-separated histograms with a narrow blue band → calibration is converged and trustworthy.
- Heavy overlap between green and red → the metric is inherently a poor discriminator on this data; consider whether your GOOD/BAD labels are consistent.
- Wide blue band → not enough samples; collect more before relying on the threshold.

ROC subplot (Level 3+). One curve per metric in a different colour, plus the diagonal $y = x$ in grey as the no-information reference. Each curve has a marker at the optimal Youden point and the metric's name + AUC in the legend, e.g. "`residual_threshold (AUC=0.94)`". AUC > 0.9 means excellent separation; AUC near 0.5 means the metric is no better than random.

Convergence subplot. Threshold value vs. number of samples seen, one line per metric. A flat tail means the estimate has stabilised; a still-trending line means more samples will continue to move the threshold.

7.3 HSV preview

The HSV calibration window stacks three panels vertically:

1. The original ROI image (top)
2. The current binary mask, white-on-black (middle)
3. The original ROI with the mask applied — only in-range pixels visible (bottom)

All three update live as the six sliders move. There is no histogram plot of pixel HSV values — the tuning is purely “does the mask look right?”.

8. Default parameter table

Aggregated from `AnalysisParams` and `AppConfig` in `usma/models.py` (lines ~910–937). This is the single source of truth for “what does USMA do out of the box?”

Parameter	Default	Range	Effect
fft_cutoff_frequency	0.07	0.02 – 0.25	Boundary between low and high spatial frequency for energy ratio
fft_energy_ratio_threshold	0.006	0.0001 – 0.1	Min HF energy fraction to flag as BAD by FFT method
lowpass_cutoff	0.07	0.03 – 0.2	Butterworth cut-off (normalised) for smooth-envelope extraction
lowpass_filter_order	7	—	Butterworth order; effectively 14 after filtfilt. Do not tune.
residual_threshold	0.005	signal-dependent	Amplitude (physical units) above which a residual sample is an exceedance
exceedance_ratio_threshold	0.7	0.2 – 0.99	Min exceedance fraction to flag as BAD by LP method
coherence_threshold	0.30	0 – 1	Max normalised coherence badness before warning
coherence_degradation_pct	0.20	0 – 1	Relative within-run badness increase that triggers degradation alert
hsv_lower	[0, 0, 0]	per-channel	HSV mask lower bound
hsv_upper	[179, 255, 240]	per-channel	HSV mask upper bound (V capped to exclude pure white)
screenshot_interval	0.25 s	—	Polling period of the monitor loop
hits_per_run	5	—	Number of hits expected per measurement
monitor_index	1	—	Which physical monitor to capture

PSD ROIs use a parallel set of analysis parameters (`psd_fft_cutoff_frequency`, `psd_fft_energy_ratio_threshold`, `psd_lowpass_cutoff`, `psd_lowpass_filter_order`) with the same defaults — they exist so PSD and FRF can be tuned independently.

9. Things this document does not cover

For completeness, these are the parts of USMA that touch data but are intentionally **out of scope** here because they are I/O or bookkeeping rather than analysis:

- The OCR pipeline (`usma/analysis/ocr.py`) — used to read run/hit/status text from the source plot. It is text recognition, not signal analysis.
- The UNV exporter (`usma/export/unv.py`) — formats results into a Universal File Format Dataset 58. It is a serialisation step, not a transformation of the data.
- The image logger (`usma/export/image_logger.py`) — saves diagnostic PNGs. No math.
- The monitor loop (`usma/monitor.py`) — schedules captures and dispatches frames; the math it triggers is everything described above.

If a future change introduces a new mathematical operation in any of these modules, it should be documented here.